

Computer Graphics

Dong, Wei
5726890

University of Warwick

September 18, 2026

1 Introduction

This project, built upon a custom rendering pipeline, implements a 3D game level with textures and a keyboard, supporting data-driven scene loading, normal mapping, basic keyboard input, and a sky sphere. Players use a first-person camera and control the game with keyboard and mouse.

<https://github.com/WeiD-Warwick/GE9M2>

1.1 Data-driven level and resource loading

The level system describes scene objects and resources through an outer configuration file.

```
1 // load Texture from filepath
2 texture {
3   ground_basecolor Src/Assets/basecolor.png
4   ground_normal Src/Assets/normal.png
5 }
6
7 // new Game Object and load component
8 scene {
9   object Player {
10    transform 0 0 0 0 0 1 1.2 2 1.2
11    component PlayerInputControllerComponent
12    component CameraComponent
13    component WeaponControllerComponent idle
14    fire reload
15    component FPSWeaponRenderComponent Src/
16    Assets/Uzi.gem fpsWeaponMaterial
17    component ColliderComponent 1.2 2 1.2
18  }
19 }
```

Listing 1: Level Text Example

During the startup phase, the engine parses configuration to load resources, create game objects, and load components via a factory system.

```
1 void load(string file) {
2   while (getline(file, line)) {
3     if (blockOpen) {
```

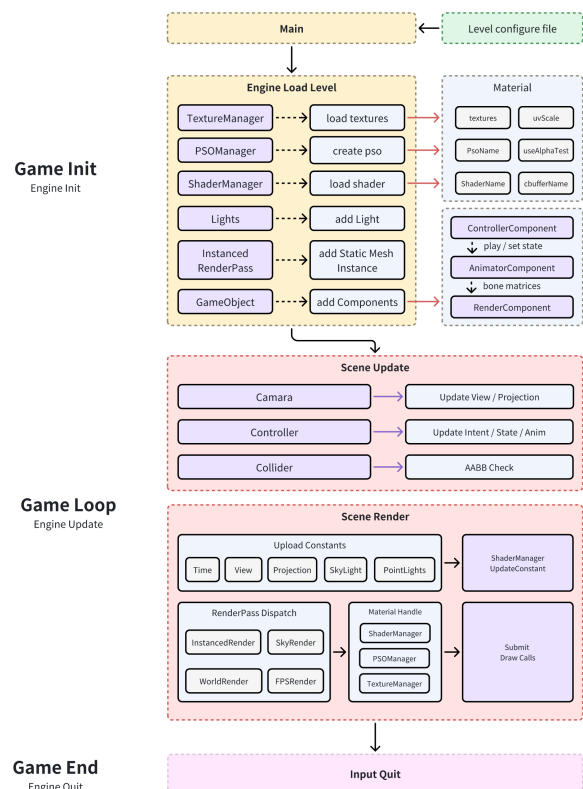


Figure 1: Architecture Design of the Project



Figure 2: Material Bind Order

```

4      openBlock(line, stack);
5  } else if (blockClose) {
6      stack.pop_back();
7  } else if (!stack.empty()) {
8      parseContent(line, stack);
9  }
10 }
11 }
12
13 void parseContent() {
14     ...
15     ctx.object = scene.createObject();
16     ctx.object->setName(ctx.name);
17     // Component creation via factory pattern
18     ComponentFactory::shared().create(
19         componentName,
20         ctx.object,
21         engine,
22         args
23     );
24 }
  
```

Listing 2: Block-based level file parsing

2 Technologies Implemented

2.1 Mesh Rendering and Materials

2.1.1 Mesh Pipeline

The rendering pipeline distinguishes static, animated, and first-person meshes using different render layers and pipeline state objects. Static meshes apply the world matrix and view projection matrix in the vertex shader, while animated meshes need to calculate skeletal transform before projection. First-person meshes are rendered in a dedicated FPS layer with a specific PSO, and all mesh types share a unified material and lighting calculation logic during the pixel shading stage.

```

1 switch (renderLayer) {
2 case RenderLayer::World:
3     cmd->SetPipelineState(
4         psoManager.get("StaticMeshPSO"));
5     break;
6 case RenderLayer::FPS:
7     cmd->SetPipelineState(
8         psoManager.get("FPSWeaponPSO"));
9     break;
10
11 case RenderLayer::Sky:
12     cmd->SetPipelineState(
13         psoManager.get("SkyPSO"));
14     break;
15 }
  
```

Listing 3: Pipeline state selection per render pass

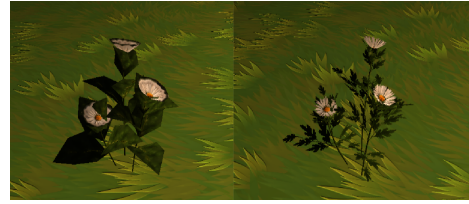


Figure 3: Difference between using alpha testing and not

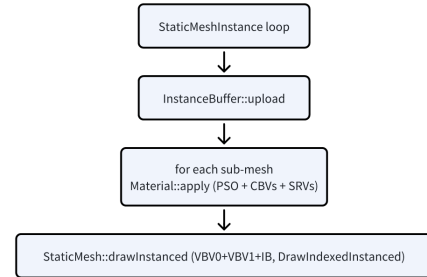


Figure 4: Instanced rendering flow

2.1.2 Alpha Testing

Alpha testing is implemented in the pixel shader by discarding pixels below a threshold. This can be used to achieve leaf effects for trees and flowers.

```

1 if (useAlphaTest && albedo.a < 0.5f) {
2     discard;
3 }
  
```

Listing 4: Per-pixel discard

2.1.3 Vertex Shader Animation

The vertex animation of the plants is done in the vertex shader, which applies displacement in the horizontal direction through a time-based sine function to simulate the effect of wind.

```

1 if (useVSAnim == 1) {
2     float windStrength = 0.15f;
3     float windSpeed = 1.2f;
4     float t = time.x * windSpeed;
5     pos.x += sin(t) * windStrength;
6 }
  
```

Listing 5: Vertex Shader Animation

2.1.4 Instanced Static Mesh Rendering

Static mesh instancing uses a single mesh but uploading per-instance transformation data of multiple instances, allowing multiple instances to be rendered directly in a single draw call.

```

1 void InstanceBuffer::upload(const InstanceData*
2     data, int count) {
3     void* mapped = nullptr;
4     _buffer->Map(0, nullptr, &mapped);
5     memcpy(mapped, data, sizeof(InstanceData) *
6         count);
7     _buffer->Unmap(0, nullptr);
  
```



Figure 5: Instanced flowers Mesh



Figure 6: Difference between using normal maps and not

```

6 }
7
8 void drawInstanced(
9     ID3D12GraphicsCommandList4* cmd,
10    D3D12_VERTEX_BUFFER_VIEW& instanceVBV,
11    int instanceCount
12 ) const {
13     // slot 0: per-vertex slot 1: per-instance
14     D3D12_VERTEX_BUFFER_VIEW vbvs[2] = { _vbView,
15         instanceVBV };
16     cmd->IASetPrimitiveTopology(
17         D3D_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
18     cmd->IASetVertexBuffers(0, 2, vbvs);
19     cmd->IASetIndexBuffer(&_ibView);
20     cmd->DrawIndexedInstanced(_numIndices,
21         instanceCount, 0, 0, 0);
22 }

```

Listing 6: DrawIndexedInstanced

2.1.5 Normal mapping

Normal mapping perturbs per-pixel normals by transforming tangent-space normals into world space using a TBN basis.

```

1 float3x3 getTBN(PS_INPUT input) {
2     float3 N = normalize(input.NormalWS);
3     float3 T = normalize(input.TangentWS);
4     float3 B = normalize(cross(N, T));
5     float3x3 TBN = float3x3(T, B, N);
6     return TBN;
7 }
8
9 // --- TBN ---
10 float3 normalWS = normalize(input.NormalWS);
11 float3x3 TBN = getTBN(input);
12 float3 mapNormal = normalTex.Sample(...).xyz;
13 mapNormal = mapNormal * 2.0 - 1.0;
14 normalWS = normalize(mul(mapNormal, TBN));

```

Listing 7: Normal Map Calculate

2.2 Camera and Player Input

2.2.1 Camera View and Projection Update

Each time when the engine update, the cameraComponent will rebuild the view matrix and projection matrix

for the current frame.

```

1 void CameraComponent::updateMatrices() {
2     view = Matrix::LookAt(
3         transform().position,
4         transform().position + transform().forward(),
5         Vec3(0, 1, 0)
6     );
7     projection = Matrix::Perspective(
8         fov, aspect, nearPlane, farPlane
9     );
10 }

```

Listing 8: Camera matrix update in CameraComponent

2.2.2 Player Input and Camera Control

Player movement is controlled by keyboard input, and the direction of movement is calculated by projecting the player's forward and rightward vectors onto a horizontal plane. Mouse input updates the camera's yaw and pitch angles in real time.

The camera is attached to the player object, with yaw applied to the player transform and pitch applied only to the camera to support stable first-person control.

```

1 yaw += dx * sensitivity;
2 pitch += dy * sensitivity;
3 pitch = clamp(pitch, -1.5f, 1.5f);
4 Quaternion qYaw = Quaternion::fromAxisAngle(
5     Vec3(0,1,0), yaw
6 );
7 transform().rotation = qYaw;

```

Listing 9: First-person control

The first-person weapon is rendered in camera space, not world space. During rendering, local rotation and offset are applied, and the view matrix is set to the identity matrix; at the same time, the camera's projection matrix is still used to ensure correct perspective and keep the weapon fixed on the screen.

```

1 Matrix W = rotateFix * offset;
2 Matrix V = Matrix::Identity();
3 Matrix P = camera->projection;
4 param.VP = V * P;

```

Listing 10: FPS weapon transform in camera space

2.2.3 Animation and Gameplay Interaction

Character animations are managed by the AnimatorComponent which updates the skeletal matrix every frame. Animation playback is driven by gameplay, rather than being directly triggered by input.

```

1 void update(float dt) {
2     controller.update(dt);
3     controller.buildSkinningMatrices();
4 }

```

Listing 11: Animator update and skinning

For the player weapon, mouse input generates firing or reloading intents, which are then processed by the weapon controller. Each intent triggers a corresponding animation, and the weapon returns to the idle state after completion.

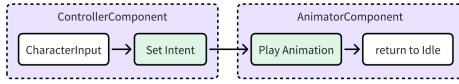


Figure 7: Animation state transition driven by input



Figure 8: State machine switching

```

1 case State::Idle:
2   if (_intent == Intent::Fire) {
3     fireRaycast();
4     animator->play(_animConfig.fire, true);
5     state = State::Firing;
6   } else if (_intent == Intent::Reload) {
7     animator->play(_animConfig.reload, true);
8     state = State::Reloading;
9   }
10  intent = Intent::None;
11  break;
  
```

Listing 12: Weapon animation state machine

2.2.4 Raycast-Based Hit Detection

When a firing intent is triggered, a raycast is emitted from the camera position along the view direction, selecting the nearest intersecting object as the target, allowing achieving precise first-person interaction.

```

1 Vec3 rayOrigin = cam->transform().position;
2 Vec3 rayDir = cam->transform().forward().normalized();
3
4 if (collider->raycast(rayOrigin, rayDir, t)) {
5   if (t < closestT) {
6     closestT = t;
7     hitObj = obj;
8   }
9 }
  
```

Listing 13: Hit Detection

2.2.5 NPC State and Animation Control

NPCs such as animals are controlled by a simple state machine, and their states switch based on their health. When they take damage, the NPC switches to a hit animation, and once health reaches zero, they will enter a death state and retain the corresponding animation.

```

1 case State::Idle:
2   if (intent == Intent::Hitted) {
3     animator->play(animConfig.hit, true);
4     state = State::Hitting;
5   } else if (intent == Intent::Death) {
6     animator->play(animConfig.death, true);
7     state = State::Death;
8   }
9   intent = Intent::None;
10  break;
  
```

Listing 14: NPCs animation state machine

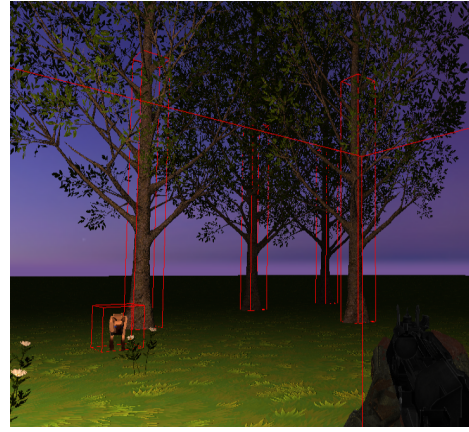


Figure 9: AABB Collision Box

2.3 Collision Detection and Response

Collision detection is implemented using axis-aligned bounding boxes (AABB). Each collider is pivoted at the bottom center of the object, supporting AABB–AABB intersection tests as well as ray-bounding box intersection queries. Each collider computes its world-space AABB by offsetting its local half extents from the object's world position. The intersection testing is performed by checking whether the minimum and maximum values of two AABBs overlap along each axis.

```

1 Vector3 minA = centerA - halfSizeA;
2 Vector3 maxA = centerA + halfSizeA;
3 Vector3 minB = centerB - halfSizeB;
4 Vector3 maxB = centerB + halfSizeB;
5
6 bool intersect =
7   (minA.x <= maxB.x && maxA.x >= minB.x) &&
8   (minA.y <= maxB.y && maxA.y >= minB.y) &&
9   (minA.z <= maxB.z && maxA.z >= minB.z);
  
```

Listing 15: AABB min/max computation and intersection test

2.3.1 Player Collision Response

Player movement uses an axially separated collision response. Movement is applied independently along the X and Z axes, once an intersection is detected, the player retracts the displacement along the corresponding axis to eliminate the collision.

```

1 // X axis
2 transform().position.x = newPos.x;
3 if (selfCollider->intersect(other)) {
4   transform().position.x = oldPos.x;
5 }
6
7 // Z axis
8 transform().position.z = newPos.z;
9 if (selfCollider->intersect(other)) {
10  transform().position.z = oldPos.z;
11 }
  
```

Listing 16: Axis-separated AABB collision response



Figure 10: Collision Debug Linebox

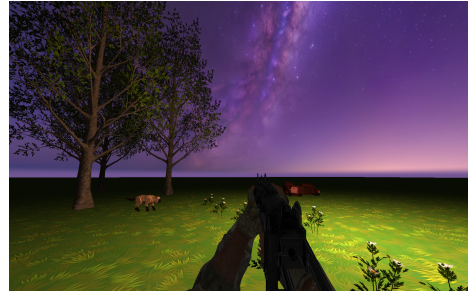


Figure 12: Sky Sphere Effect

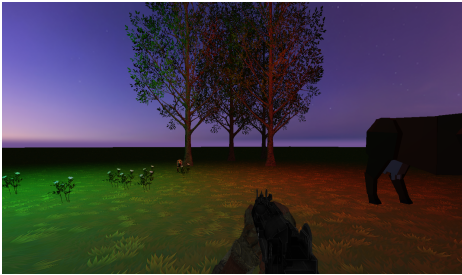


Figure 11: Configured Lights in red and green.

2.3.2 Raycast-Based Collision Query

Ray-AABB intersection is used for hit detection in game-play interactions. A ray is cast from the camera along the view direction, and select the nearest intersecting collider as the hit target, allowing supporting precise player-NPC interactions such as weapon shooting.

```
1
2 if (collider->raycast(rayOrigin, rayDir, t)) {
3     // hit
4 }
```

Listing 17: Ray-AABB intersection Check

2.3.3 Collision Debug Visualization

For easier debugging, colliders can be rendered as wire-frame boxes in a dedicated debug render layer.

2.4 Lighting System

2.4.1 Game Lighting

Lighting is calculated by a Lambertian diffuse model. Normal mapping is applied per pixel, and accumulating lighting contributions from a constant sky light and multiple point lights.

```
1 cbuffer CB {
2     float skyLightIntensity;
3     float3 skyLightColor;
4     int pointLightCount;
5     float4 lightPosWS[MAX_POINT_LIGHTS];
6     float4 lightColor[MAX_POINT_LIGHTS];
7     float4 lightParams[MAX_POINT_LIGHTS];
8 };
9
10 // --- Sky Light ---
11 float3 lighting =
12     albedo * skyLightColor * skyLightIntensity;
```

```
13
14 // --- Point Light ---
15 for (int i = 0; i < pointLightCount; ++i) {
16     float3 L = lightPosWS[i].xyz - input.PosWS.xyz;
17     float dist = length(L);
18     float3 lightDir = normalize(L);
19
20     // attenuation
21     float attenuation =
22         saturate(1.0 - dist/lightParams[i].x);
23     attenuation *= attenuation;
24
25     // Lambert
26     float NdotL =
27         max(dot(normalWS, lightDir), 0.0);
28     float3 diffuse =
29         (albedo/PI) * lightColor[i].xyz * NdotL;
30     lighting += diffuse * lightParams[i].y *
31         attenuation;
32 }
```

Listing 18: Game Lighting

2.5 Visual Effects and Enhancements

2.5.1 Sky Sphere Rendering

The sky sphere is rendered in a dedicated Sky layer as a background effect. Its view matrix removes translation components, rotating only with the camera and not moving with position. The sky sphere is rendered using a specialized PSO with depth writing disabled, ensuring it always remains behind the scene geometry.

```
1 Matrix withoutTranslation() const {
2     Matrix r = *this;
3     r.a[0][3] = 0.0f;
4     r.a[1][3] = 0.0f;
5     r.a[2][3] = 0.0f;
6     return r;
7 }
8
9 void onRender(RenderContext& renderContext) {
10     MaterialParam param;
11     param.W =
12         Matrix::Translation(cam->transform().position);
13     param.VP =
14         cam->view.withoutTranslation() * cam->projection;
15     drawMeshes(renderContext, param);
16 }
```

Listing 19: View matrix without translation for sky rendering

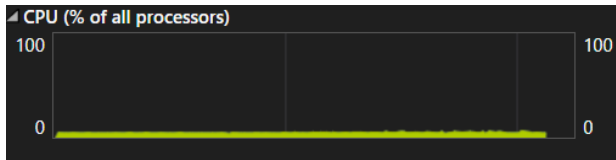


Figure 13: CPU Total Usage

entire pipeline and my superficial use of DirectX 12. Initially, I wanted to design the project more engineered. However, due to my shallow understanding of DirectX 12, I later found that the designed components couldn't be reused when implementing instantiation, which was a bit of a disappointment.

Function Name	Total CPU [unit, %]	Self CPU [unit, %]
[External Call] ucrtbased.dll!0x00007fb041e244e	11882 (27.88%)	11882 (27.88%)
[External Call] d3d12sdklayers.dll!0x00007fb03102a0	3122 (7.33%)	3122 (7.33%)
stbi_image_decode	6472 (15.19%)	2158 (5.06%)
[External Call] ucrtbased.dll!0x00007fb041d495	1915 (4.49%)	1915 (4.49%)
[External Call] ntdll.dll!0x00007fc1898c53c	42579 (99.92%)	1845 (4.33%)

Function Name	Total CPU [unit, %]	Self CPU [unit, %]
GERM2 (PID: 86472)	42613 (100.00%)	0 (0.00%)
[System Code] ntdll.dll!0x00007fc1898c53c	42579 (99.92%)	1845 (4.33%)
WinMainCRTStartup	40734 (95.59%)	0 (0.00%)
__scrt_common_main	40734 (95.59%)	0 (0.00%)
__scrt_common_main_seh	40734 (95.59%)	0 (0.00%)
invoke_main	40734 (95.59%)	0 (0.00%)
WinMain	40734 (95.59%)	2 (0.00%)
Engine:update	21919 (51.44%)	2 (0.00%)
Engine:Engine	18345 (43.05%)	0 (0.00%)

Figure 14: CPU Time

3 Evaluation

3.0.1 Performance

Overall, CPU usage remains low and stable throughout execution, indicating that the engine is not CPU-bound under this workload.

Most CPU time is spent in system and kernel-level calls, while the engine update loop contributes little self CPU time, indicating lightweight per-frame logic.

3.1 Stability

The core system is running normally, and the rendering pipeline is performing as expected. There are no major crashes or logical errors.

4 Limitations

Although I used ComPtr to avoid memory leaks, the chaotic management of ownership still caused many problems.

Furthermore, the initial structure did not adequately address the issue of instantiated meshes, which means I may need to redesign the collision system to support instantiated meshes.

5 Conclusion

This course was a crucial opportunity for me to begin gaining a deeper understanding of game engines. I not only learned about pipeline development but also experienced real-world project development firsthand, rather than just simple demos or practice projects. The assignment revealed my limited understanding of the